ContractForge: Why Specs Fail After L0

Common failure patterns across 18 SRS documents, abstracted away from any single SRS

Bavya Sri Kilari (2025701029), Akshith K (2024111004), Vivek R (2019112020)
Autonomous Software Engineering, Monsoon 2026, IIIT Hyderabad

13 November 2026 Repository: github.com/Ohshit-ak/AUTOSE-Leaniers

1 Summary

We went through every run on the 18 SRS documents: L0 (`qwen-v10.1`, 718 specs) and the per-spec L1/L2 chain (`qwen-chain-v4`, strict and `-fallback-single`, 3 repetitions each, 1452 spec-runs per setting). We also re-ran Lean on the 102 saved L0 specs that do not compile. The aim is a rough, SRS-independent picture of where and why specs stop, not a refined per-document analysis.

- **After L0, almost nothing fails to *compile*.** Of 1452 L0-accepted spec-runs (strict), 419 do not verify end to end: about 113 stop at L1 and about 306 at L2. Of the 314 failing L2 components, **285 are PO3 failures**: the code compiles, but Lean cannot prove that it satisfies the contract. Real Lean errors after L0 (unknown field or name, type, syntax) are only about 40 in total.
- **L2 synthesis is the bottleneck, not L1.** With `-fallback-single` the L1 failures disappear, but PO3 failures rise from 285 to 327: the specs that could not be split now reach L2 and fail there.
- **The same five failure modes appear at every layer** (Section 3): names outside the declared vocabulary, copying instead of computing, relational `post` conditions turned into no computation, spec defects that L0 lets through, and a repair loop that does not converge.
- **L0's own compile failures** come almost entirely from free-prose requirements (107 of 111); templated data constraints and use cases almost never fail (Section 4).

In one line: the model treats the spec as text to echo, not a constraint to satisfy.

2 Where specs stop

	strict	fallback
spec-runs into L1/L2 (18 SRS × 3 reps)	1452	1452
routed single-step (no split)	1305	1305
needs a split	147	147
fully verified end to end	**1033**	**1098**
not verified	419	354
L1 failures	113	0
L2 failing components	314	361
of which PO3 (compiles, does not satisfy contract)	**285**	**327**
of which Lean errors (field, name, type, syntax)	29	34

Table 1: Totals over all 18 SRS documents, `qwen-chain-v4`. L0 is frozen at `qwen-v10.1/rep1`.

L1 failure class (strict)		L2 failure class (strict)	
identity (component only copies)	**64**	po3	**285**
po2 (chain does not refine Spec)	31	unknown-field	11
visibility (reads a record it cannot see)	8	unknown-name	9
unknown-field	6	type-error	7
type-error	4	syntax	2

Table 2: Failure classes from `harness/chain_failures.py`. 302 of the 314 L2 failures are in the first (often the only) component.

3 Five failure modes that repeat across layers

3.1 1. Names outside the declared vocabulary

The model writes against a vocabulary it imagines, not the one in the file.

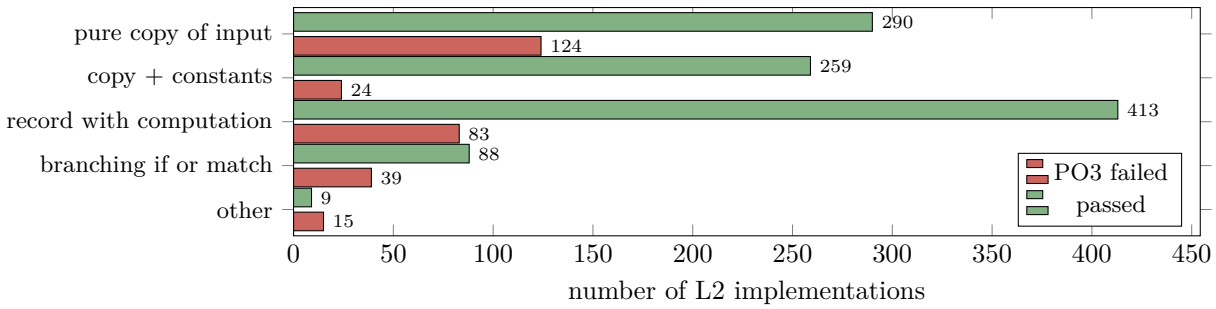
- **L0:** fields used in `pre`/`post` but never declared in `Input`/`Output`, and library functions that do not exist (`List.last`, `String.substring`, dot-notation such as `.head` on a lambda). Together about half of all L0 compile failures.

- **L1:** `unknown-field` and `visibility`: the second contract reads an `Input` field that is not in `Mid`.
- **L2:** `unknown-field` / `unknown-name`: a record field that the locked result structure does not have.

3.2 2. Copy instead of compute

- **L1:** the most common L1 failure is `identity` (64 of 113): one component only passes its input through.
- **L2:** about half of the PO3 failures are pure copies of the input record, or copies plus constants:

```
def SpecImpl (x : Input) : Output :=
  { mode := x.mode, antenna := x.antenna }      -- the contract asks for a value that depends on x.mode
```



Caveat. About 27% of *passing* implementations are also pure copies. Copying does not cause failure by itself: the model copies by default, and a copy passes only when the spec is weak enough to accept it (see the `spec_check.trivial` flag: 584 of 1033 verified spec-runs have a copy-and-constants witness).

3.3 3. A relational `post`, but the model must write a function

When PO3 fails, the harness names the conjuncts of `post` that the function does not establish. We sorted the 380 such conjuncts by syntactic form (one conjunct can have several forms, so the shares overlap):

form of the failing conjunct	share
arithmetic / length / order (`+`, `<`, `.length`)	∼39%
output must equal a specific string literal	∼39%
bi-implication (`o.x = i.x` ↔ `cond`)	∼35%
output must be a fixed flag or option value (`= none`, `= true`)	∼34%
quantifier over a collection (∀ `e` ∈ `o.events, ...`)	∼33%
input-dependent conditional (`i.mode = "X"` → `o.f = Y`)	∼19%
list property (`Nodup`, sorted, filtered)	∼11%

Every one of these needs a case split or a real computation (filter, count, compare, branch on an input value). The model skips it and returns the input. 212 of 285 PO3 failures miss exactly one conjunct, so the function is usually close, but the missing part is the part that carries the requirement.

3.4 4. Spec defects that L0 lets through

39 of the 380 failing conjuncts do not mention the output at all, for example:

```
i.count ≤ 3                              -- a fact about the input, not implied by pre
∃ a ∈ i.accounts, a.username = i.username
List.Pairwise (fun a b => a.date ≤ b.date) i.events
```

No implementation can establish these, so the fault is in L0, not L2. The L0.5 checks test that `post` rejects a changed output, but not that every conjunct of `post` constrains the output. The opposite defect is also common: weak specs that a pure copy satisfies, which pass L2 without the code doing anything the requirement asks for.

3.5 5. The repair loop does not converge

- **L2:** 143 of the 285 PO3 failures go through all five attempts at the same `check` stage and fail each time, although the feedback names the exact conjunct that does not hold.
- **L0:** retries swing between name errors and type errors (`names` → `types` → `names`), or hit the same syntax error five times out of five (8 specs) or the same name error five times out of five (12 specs).
- Failures are stable across repetitions: most failing specs fail in all three. All repetitions read the same L0 run at temperature 0.2, so low sampling variance explains part of this; we report it only as "failures are stable", not as proof that the spec is always at fault.

4 L0's own compile failures

111 of 718 L0 specs do not compile (rep1 of each SRS). **107 of the 111 are free-prose requirements**; data constraints (3) and use cases (1), which go through templates, almost never fail. The less structured the source sentence, the more likely the spec does not compile.

first Lean error (re-run of the 102 saved files)	specs	%
names outside the vocabulary: undeclared `Input`/`Output` field, invented library function	49	48
Lean typing: Bool vs Prop, partial `head` used as total, a number where a structure is expected, "function expected"	12	12
syntax from other languages (∈ in binders, stray tokens, `matches`, `repeat`)	12	12
typeclass: missing `Decidable`, `SDiff (List String)`, `Membership String String`, stuck `Repr`	12	12
the spec's own example: `decide` fails, or proves the example violates `pre`/`post`	11	11
other (array index proof, missing record fields, `{..}` without expected type)	5	5

Table 3: One spec timed out or compiled on re-run and is not counted. Final repair stages recorded by the harness agree: names 37, types 34, syntax 22, proof 15.

5 What this suggests

- **Check that every `post` conjunct mentions the output** (or is implied by `pre`) at L0.5. This removes the unrealizable specs before they reach L2.
- **Treat a copy-satisfiable spec as weak at L0**, not only as a flag in the summary; otherwise "verified" includes code that does nothing.
- **At L2, give the model the failing conjunct as a case to implement**, e.g. "add a branch on `i.mode`", rather than restating it; the current feedback is ignored in about half of the PO3 failures.
- **Constrain the vocabulary:** list the declared fields and the allowed library functions in the prompt, and map common invented names (`List.last` → `List.getLast?`) in the fixups.
- **Template more of the free-prose requirements**, since templated kinds almost never fail at L0.